UNIVERSIDADE PAULISTA – UNIP
CIÊNCIA DA COMPUTAÇÃO – 8º SEMESTRE

BRUNO BONAVIGO DACAL – N309EC4
LUCAS DE ARAUJO VERGARA – N090758
VITOR OLIVEIRA DOS SANTOS – N083DG1

CONTROLE INTELIGENTE DA ILUMINAÇÃO PÚBLICA A PARTIR DE DADOS URBANOS JÁ EXISTENTES

Guia do Usuário e Manual de Operação do Conjunto Prototipado (Entregável 2)
Vertente SmartCity

SÃO PAULO
2026

BRUNO BONAVIGO DACAL – N309EC4
LUCAS DE ARAUJO VERGARA – N090758
VITOR OLIVEIRA DOS SANTOS – N083DG1

CONTROLE INTELIGENTE DA ILUMINAÇÃO PÚBLICA A PARTIR DE DADOS URBANOS JÁ EXISTENTES

Guia do Usuário e Manual de Operação do Conjunto Prototipado (Entregável 2)

Documento técnico apresentado ao curso de Bacharelado em Ciência da Computação da Universidade Paulista – UNIP como Entregável 2 da qualificação do Trabalho de Conclusão de Curso (vertente SmartCity: telemetria viária e infraestrutura urbana).

Orientador: Prof. Marco Gomes

SÃO PAULO

2026

SUMÁRIO

1 IDENTIFICAÇÃO DO ECOSSISTEMA E VERTENTE 5
1.1 Vertente do projeto 5
1.2 Título da solução e equipe executora 5
1.3 Visão geral do funcionamento 5
1.4 Diagrama de arquitetura de alto nível 6
2 MANUAL DE INSTALAÇÃO E MONTAGEM DO HARDWARE 8
2.1 Lista de materiais (BOM) 8
2.2 Esquema de ligação e pinagem 9
2.3 Especificações de alimentação elétrica 10
2.4 Instruções de fixação mecânica 11
3 GUIA DE INSTALAÇÃO E CONFIGURAÇÃO DO SOFTWARE 12
3.1 Requisitos mínimos de sistema e dependências 12
3.2 Configuração da borda (firmware do ESP32) 12
3.2.1 Gravação (flashing) do firmware 12
3.2.2 Credenciais de rede e parâmetros MQTT 13
3.3 Subida do servidor, back-end e banco de dados 14
3.3.1 Na nuvem (VM Oracle), uma única vez 14
3.3.2 Variáveis de ambiente 14
3.3.3 No computador (ambiente local de desenvolvimento) 15
3.4 Instalação da aplicação cliente (painel) 15
4 SEÇÃO ESPECIAL – MÓDULO DE INTELIGÊNCIA ARTIFICIAL 17
4.1 Objetivo operacional da IA 17
4.2 Dataset de treinamento e validação 17
4.2.1 Origem dos dados 17
4.2.2 Volume amostral e pré-processamento 18
4.3 Arquitetura do modelo e treinamento 18
4.4 Resultados e métricas empíricas 19
4.5 Declaração formal de uso de IA assistiva 21
5 MANUAL DE OPERAÇÃO E DIAGNÓSTICO DO CONJUNTO PROTOTIPADO 22
5.1 Procedimento de partida e calibração 22
5.2 Instruções de operação diária 22
5.3 Guia de resolução de problemas 24
5.4 Desligamento seguro e manutenção preventiva 25
5.4.1 Desligamento 25
5.4.2 Manutenção preventiva 25

REFERÊNCIAS.....27

(Se o sumário aparecer vazio, clique nele e pressione F9 no Word para atualizar.)

1 IDENTIFICAÇÃO DO ECOSSISTEMA E VERTENTE

1.1 Vertente do projeto

Este trabalho pertence à vertente SmartCity (Cidades Inteligentes e Infraestrutura Urbana), no eixo de telemetria viária. O domínio de aplicação é a iluminação pública de vias de pedestres do município de São Paulo: o sistema regula o nível de luz de cada poste a partir de dados urbanos que a cidade já produz (volume de pessoas e ocorrências por trecho de via, observados por câmeras), sem exigir sensores novos no poste. Não se trata de aplicação genérica: os trechos de via e os postes vêm do cadastro público GeoSampa, os níveis de luz seguem as classes P3 a P5 da ABNT NBR 5101 e o recorte de demonstração é real (bairro Jardim Helena, zona leste).

1.2 Título da solução e equipe executora

Título: Controle inteligente da iluminação pública a partir de dados urbanos já existentes: proposta de integração entre plataforma de monitoramento e telegestão no município de São Paulo.

Quadro 1 – Equipe executora e funções no projeto

Integrante	RA	Função no projeto
Vitor Oliveira dos Santos	N083DG1	Software (firmware, serviço de decisão, API, banco, painel, implantação na nuvem) e montagem do nó de bancada
Bruno Bonavigo Dacal	N309EC4	Documentação e apoio à pesquisa
Lucas de Araujo Vergara	N090758	Documentação e apoio à pesquisa

Fonte: Elaborado pelos autores (2026).

Orientador: Prof. Marco Gomes. **Repositório:** github.com/oliveiravitor32/TCC2 (pasta `Codigo/`).

1.3 Visão geral do funcionamento

O fluxo de ponta a ponta é: (1) a plataforma de câmeras publica, por trecho de via, o volume médio de pessoas a cada 5 min ou uma ocorrência (segurança ou trânsito) na interface REST `POST /eventos`; (2) o serviço de decisão, na nuvem, estima a movimentação da próxima hora com um modelo XGBoost, decide o nível de luz com inferência nebulosa Mamdani e passa a decisão por uma camada de verificação que impõe o piso (classe P5 = 0,5 do pleno) e a taxa máxima de redução

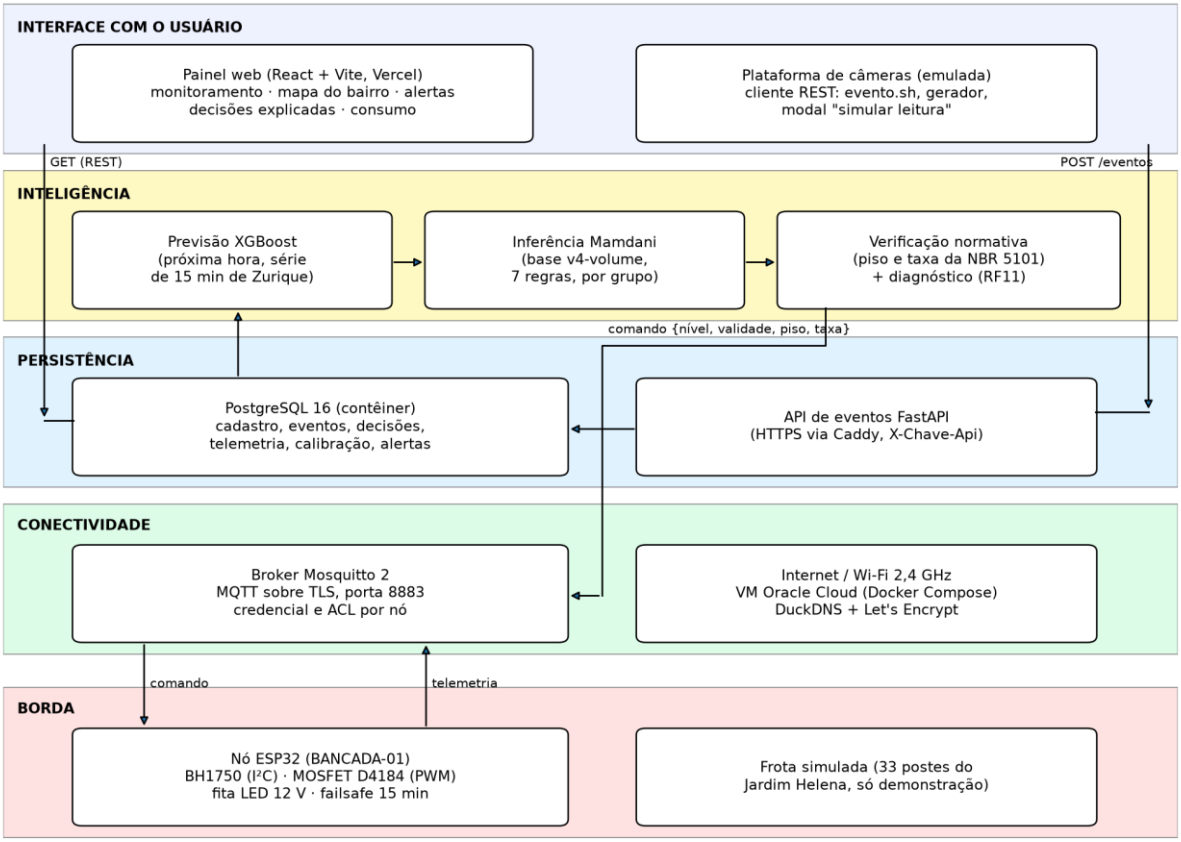
(0,1 do pleno por minuto) da NBR 5101; (3) o comando sai por MQTT com TLS até o nó ESP32; (4) o nó converte o nível em ciclo de trabalho PWM pela sua tabela de calibração, ajusta a fita de LED, mede a iluminância com o sensor BH1750 e devolve a telemetria; (5) tudo fica gravado no PostgreSQL e aparece no painel web.

O nível de luz é sempre uma **fração do fluxo pleno** (1,0 = 10 lux, classe P3; 0,5 = 5 lux, classe P5). A nuvem nunca envia duty, tensão ou lux: quem converte nível em sinal elétrico é o nó.

1.4 Diagrama de arquitetura de alto nível

A Figura 1 mostra o ecossistema prototipado separado nas cinco camadas pedidas: Borda, Conectividade, Persistência, Inteligência e Interface com o Usuário.

Figura 1 – Arquitetura de alto nível do ecossistema prototipado, por camada



Fonte: Elaborado pelos autores (2026).

Quadro 2 – Camadas do ecossistema e componentes

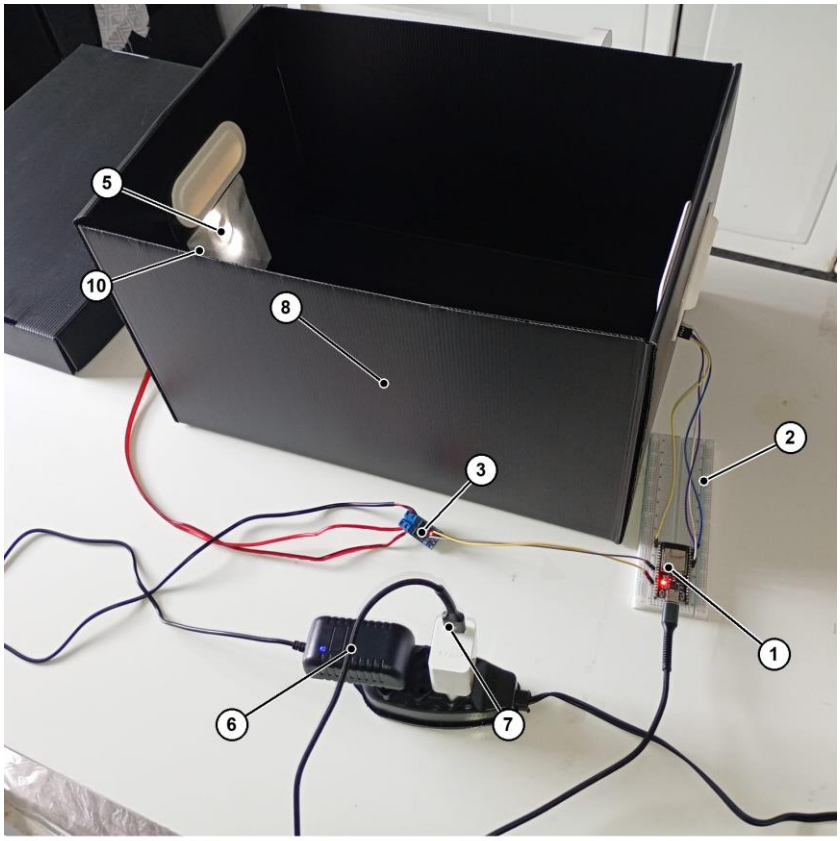
Camada	**Componentes**	**Tecnologia e protocolo**
Borda	Nó de bancada BANCADA-01 (ESP32 + BH1750 + MOSFET + fita LED); frota simulada de 33 postes (só demonstração)	C++/Arduino (core esp32 3.3.x); I²C (sensor), GPIO/PWM 5 kHz 12 bits (acionamento)
Conectividade	Wi-Fi 2,4 GHz; broker Mosquitto 2 na VM; Caddy (HTTPS)	TCP/IP; MQTT 3.1.1 sobre TLS na porta 8883, credencial e ACL por nó; HTTPS 443 com certificado Let's Encrypt
Persistência	PostgreSQL 16 em contêiner; API de eventos (FastAPI)	SQL (11 migrações); REST/JSON
Inteligência	Previsão XGBoost; inferência Mamdani (scikit-fuzzy); verificação normativa; diagnóstico de falha	Python 3.12; serviço sempre ligado (`decisao.servico`)
Interface com o usuário	Painel web (monitoramento, mapa do bairro, alertas, decisões explicadas, consumo); clientes que publicam eventos	React 19 + Vite + Tailwind 4, publicado na Vercel; HTTP/REST

Fonte: Elaborado pelos autores (2026).

2 MANUAL DE INSTALAÇÃO E MONTAGEM DO HARDWARE

O nó de bancada representa um poste de luz em escala reduzida. Ele fica dentro de uma caixa escura (anteparo), com a fita de LED numa parede e o sensor de luz na parede oposta, para que a leitura seja só da luz produzida pelo nó. A Figura 2 mostra o nó montado.

Figura 2 – Nó de bancada montado (marcações numeradas)



1	ESP32 (controlador do nó)	**2**	Protoboard
3	Módulo MOSFET D4184 (aciona a fita)	**5**	Fita de LED de 12 V (atrás do difusor)
6	Fonte de 12 V da fita	**7**	Fonte USB do ESP32
8	Caixa de polipropileno	**10**	Papel difusor da luz da fita

Fonte: Acervo dos autores (2026).

2.1 Lista de materiais (BOM)

Quadro 3 – Lista de materiais do nó de bancada

Componente	Modelo / part number	Tensão de operação	Função no circuito
Microcontrolador	ESP32 DevKit V1 (DOIT), 38 pinos, módulo ESP32-WROOM-32	5 V na USB; lógica 3,3 V	Wi-Fi, sessão MQTT/TLS, recebe o comando, gera o PWM, lê o sensor e publica a telemetria
Sensor de iluminância	BH1750FVI no módulo GY-302 (endereço I²C 0x23)	3,3 V (do ESP32)	Mede a luz entregue no plano de medição, em lux (modo de alta resolução 2, 0,11 lx)

Componente	Modelo / part number	Tensão de operação	Função no circuito
Módulo de acionamento	Módulo MOSFET D4184 com entrada PWM, bornes a parafuso	Lógica a partir de 3,3 V; carga de 5 V a 30 V	Transforma o PWM do ESP32 em variação de corrente na fita
Fonte de luz	Fita LED 5050 branca, 60 LEDs/m, 14,4 W/m, 2 fios; segmento de 3 LEDs (6 cm)	12 V CC	Representa a luminária do poste (0,86 W no pleno)
Fonte de alimentação	Fonte chaveada 12 V / 2 A	Entrada 100–240 V CA; saída 12 V CC	Alimenta só o ramo de iluminação
Cabo USB	Micro-USB de dados	5 V	Alimenta o ESP32 e serve para gravar o firmware e para a conferência pela serial
Protoboard e jumpers	Protoboard de 830 pontos; jumpers macho-macho e macho-fêmea	—	Ligações de sinal sem solda
Anteparo	Caixa de polipropileno preto fosco 43,7 × 31 × 24 cm; difusor de papel	—	Isola o sensor da luz ambiente; fita e sensor a 0,437 m

Fonte: Elaborado pelos autores (2026).

2.2 Esquema de ligação e pinagem

O ESP32 fica na protoboard com o conector USB para cima (as legendas dos pinos ficam embaixo da placa). As ligações são as dos Quadros 4 e 5 e da Figura 3.

Quadro 4 – Pinagem ESP32 → sensor BH1750 (GY-302)

Pino do ESP32	Pino do BH1750	Sinal
3V3	VCC	Alimentação 3,3 V
GND	GND	Terra comum
GPIO21	SDA	Dados I²C
GPIO22	SCL	Relógio I²C
— (não ligado)	ADDR	Aberto ou em GND: endereço 0x23

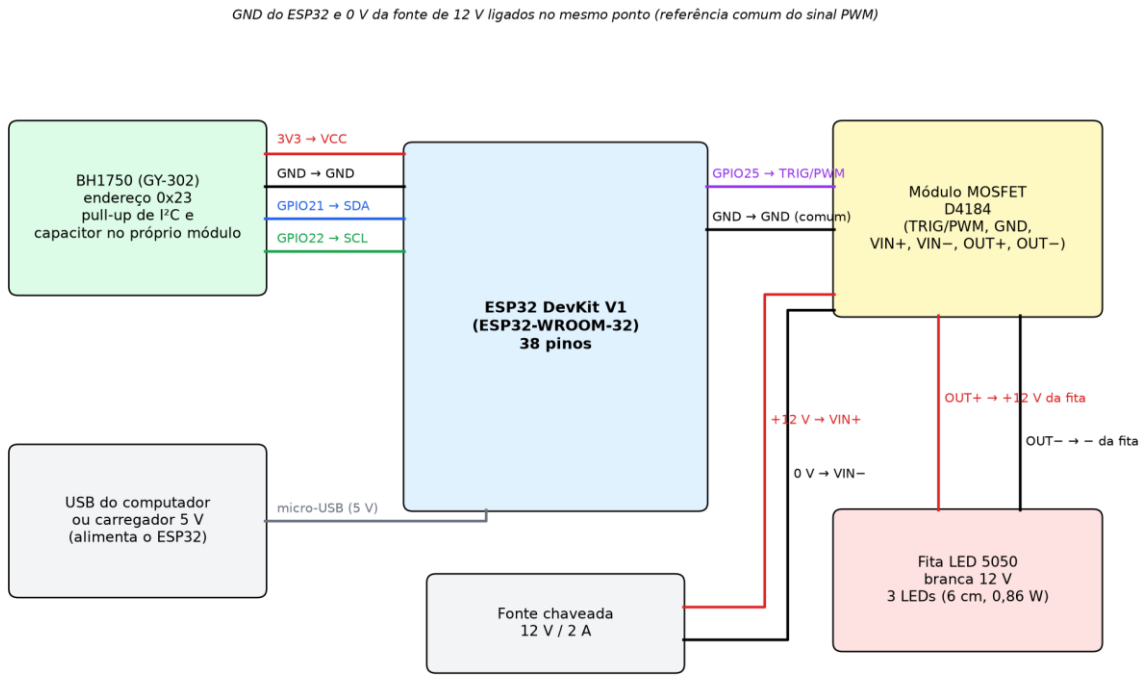
Fonte: Elaborado pelos autores (2026).

Quadro 5 – Pinagem ESP32 → módulo MOSFET D4184 → fita e fonte

Origem	Destino	Sinal
ESP32 GPIO25	D4184 TRIG/PWM	PWM 5 kHz, 12 bits (0 a 4095)
ESP32 GND	D4184 GND (sinal)	Referência comum do PWM
Fonte 12 V (+)	D4184 VIN+	Alimentação da carga
Fonte 12 V (−)	D4184 VIN−	Retorno da carga
D4184 OUT+	Fita LED (+12 V)	Saída para a fita
D4184 OUT−	Fita LED (−)	Retorno da fita, chaveado pelo MOSFET

Fonte: Elaborado pelos autores (2026).

Figura 3 – Esquema elétrico das ligações do nó



Fonte: Elaborado pelos autores (2026).

Resistores e capacitores. O circuito não precisa de componentes discretos adicionais: o módulo GY-302 já traz os resistores de pull-up do barramento I²C e o capacitor de desacoplamento do sensor; o D4184 já traz o resistor de porta e a proteção do MOSFET; a fita de LED traz os resistores de limitação de corrente (um a cada três LEDs). Por isso não se usa resistor de pull-down no GPIO25: o firmware configura o pino como saída PWM logo na partida.

Cuidado. O GND do ESP32 precisa estar ligado ao GND do módulo MOSFET; sem essa referência comum, o PWM não aciona o MOSFET e a fita fica apagada ou piscando. Nunca ligue os 12 V em nenhum pino do ESP32.

2.3 Especificações de alimentação elétrica

Quadro 6 – Alimentação do nó

Ramo	Tensão nominal	Corrente de pico	Fonte e dimensionamento
Lógica (ESP32 + BH1750)	5 V na USB; 3,3 V regulado na placa	Cerca de 240 mA nos picos de transmissão Wi-Fi do ESP32 (valor típico do fabricante); BH1750 abaixo de 0,2 mA	Porta USB do computador ou carregador de 5 V / 1 A (folga de mais de 4 vezes)
Iluminação (fita)	12 V CC	Cerca de 72 mA no pleno (0,86 W ÷ 12 V); limite de projeto < 300 mA	Fonte chaveada 12 V / 2 A: folga de mais de 20 vezes

Fonte: Elaborado pelos autores (2026).

Considerações térmicas. A potência total do nó é inferior a 2 W. O MOSFET do D4184 trabalha muito abaixo da sua corrente nominal e não aquece de forma perceptível; o segmento de 6 cm de fita dissipa menos de 1 W. Mesmo assim, mantenha a fonte de 12 V fora da caixa (ela aquece levemente e influenciaria a leitura se ficasse dentro) e não cubra a fita com material inflamável além do difusor de papel, que fica afastado dela.

2.4 Instruções de fixação mecânica

1) Fixe a fita de LED no centro de uma das paredes menores da caixa, pelo adesivo do próprio produto, com os LEDs voltados para dentro.
2) Fixe o difusor de papel diante da fita, sem tocá-la.
3) Fixe o sensor BH1750 no centro da parede oposta, com a face sensível voltada para a fita (distância de 0,437 m entre os dois).
4) Passe os fios por um furo pequeno na caixa e vede a folga com fita isolante preta, para não entrar luz do ambiente.
5) Deixe o ESP32, a protoboard e o módulo MOSFET fora da caixa, sobre superfície isolante e seca; a fonte de 12 V também fica fora.
6) Antes de usar, feche a tampa e confira que a leitura com a fita apagada (residual) fica abaixo de 5 % do valor da classe (0,5 lux); isso é feito pela conferência da seção 5.1.

Intempéries e aterramento. O nó é um protótipo de bancada para ambiente interno e não tem proteção contra chuva. Todo o circuito é de baixa tensão em corrente contínua (12 V e 5 V), isolado da rede elétrica pela fonte chaveada, por isso não há aterramento de proteção no nó; o único ponto em 127/220 V é a entrada da fonte, que já vem fechada de fábrica. Numa instalação real o equivalente do nó seria o controlador de telegestão do próprio poste, que já tem invólucro, proteção contra surtos e aterramento definidos pelo Caderno de Encargos da concessão.

3 GUIA DE INSTALAÇÃO E CONFIGURAÇÃO DO SOFTWARE

3.1 Requisitos mínimos de sistema e dependências

Quadro 7 – Requisitos de sistema

Onde	Requisito	Versão usada no projeto
Servidor (nuvem)	Linux com Docker e Docker Compose v2	Ubuntu 24.04 LTS ARM (Oracle Cloud, VM.Standard.A2.Flex, camada gratuita); Docker 24+ (pacotes docker.io e docker-compose-v2)
Imagem dos serviços Python	Python 3.11 ou superior	python:3.12-slim (Dockerfile da pasta Codigo)
Banco de dados	PostgreSQL	postgres:16 (contêiner)
Broker MQTT	Mosquitto	eclipse-mosquitto:2 (contêiner)
Proxy HTTPS	Caddy	caddy:2 (contêiner)
Computador de desenvolvimento	Windows 10/11, Git, Docker Desktop, Python 3.12, Node.js 20 ou superior	Windows 10; Node.js 24; Python 3.12 em ambiente virtual `Codigo/.venv`
Firmware	Arduino IDE 2 com o core esp32 da Espressif	Core esp32 3.3.x; biblioteca BH1750 (Christopher Laws); placa "DOIT ESP32 DEVKIT V1"
Painel	Node.js e npm	React 19, Vite 8, Tailwind 4, Leaflet 1.9.4, react-leaflet 5.0.0

Fonte: Elaborado pelos autores (2026).

As dependências Python estão declaradas uma única vez em `Codigo/pyproject.toml` (numpy, scipy, scikit-fuzzy 0.5.0, FastAPI, uvicorn, psycopg 3, paho-mqtt 2, pandas, XGBoost 3, scikit-learn) e as de teste em `Codigo/requirements-dev.txt` (pytest, httpx, pyserial). O `Dockerfile` instala exatamente essa lista. As do painel estão em `Codigo/painel/package.json`.

Para preparar o ambiente Python no computador (PowerShell, pasta `Codigo`):

```
python -m venv .venv
.venv\Scripts\pip install -r requirements-dev.txt
.venv\Scripts\python -m pytest
```

3.2 Configuração da borda (firmware do ESP32)

3.2.1 Gravação (flashing) do firmware

1) Instale a Arduino IDE 2. Em Arquivo > Preferências, adicione a URL de placas da Espressif e, no Gerenciador de Placas, instale "esp32 by Espressif Systems" versão 3.3.x.

2) No Gerenciador de Bibliotecas, instale "BH1750" (Christopher Laws). As demais bibliotecas (WiFi, Wire, Preferences e o cliente MQTT `mqtt_client.h` do ESP-IDF) já vêm com o core.
3) Abra `Codigo/firmware/no/no.ino`. Copie `segredos.exemplo.h` para `segredos.h` na mesma pasta e preencha (item 3.2.2).
4) Ligue o ESP32 pela USB. Em Ferramentas, escolha a placa "DOIT ESP32 DEVKIT V1" e a porta (no computador do projeto, COM3).
5) Clique em Carregar. Se a gravação não começar, segure o botão BOOT da placa até aparecer "Connecting...".
6) Abra o Monitor Serial a 115200 baud: devem aparecer a conexão ao Wi-Fi, a conexão ao broker e o estado "online".

3.2.2 Credenciais de rede e parâmetros MQTT

Tudo o que é segredo do nó fica em `segredos.h`, que **não vai para o repositório** (está no `.gitignore`). Os campos são:

```
#define WIFI_SSID    "nome-da-rede-2.4GHz"
#define WIFI_SENHA   "senha-da-rede"
#define MQTT_URI     "mqtts://tcc-iluminacao-publica.duckdns.org:8883"
#define MQTT_USUARIO "BANCADA-01"
#define MQTT_SENHA   "senha-gerada-pelo-preparar.sh"
static const char *MQTT_CA_PEM = R"(-----BEGIN CERTIFICATE-----
...certificado da CA mostrado pelo preparar.sh...
-----END CERTIFICATE-----
)";
#define PONTO "BANCADA-01"
```

Quadro 8 – Parâmetros MQTT do nó

Parâmetro	**Valor**
Broker	tcc-iluminacao-publica.duckdns.org (VM); no desenvolvimento local, o IP do computador na rede (nunca "localhost")
Porta	8883 com TLS (VM); 1883 sem TLS só no ambiente local
Versão do protocolo	MQTT 3.1.1, keepalive de 30 s
Autenticação	Usuário = código do ponto (ex.: BANCADA-01) e senha própria; a ACL só deixa o nó ler o próprio comando e escrever a própria telemetria e estado
Tópico assinado	`iluminacao/{ponto}/comando` (QoS 1): nível, validade, piso, taxa e `decisao_id`
Tópicos publicados	`iluminacao/{ponto}/telemetria` (QoS 1): duty, lux, nível, nivel_antes, lux_antes, decisao_id; `iluminacao/{ponto}/estado` (retido): online, offline (testamento) ou manutencao

Fonte: Elaborado pelos autores (2026).

A rede Wi-Fi precisa ser de 2,4 GHz (o ESP32 não conecta em 5 GHz). Na apresentação, pode-se usar o roteador do celular (hotspot) configurado em 2,4 GHz.

3.3 Subida do servidor, back-end e banco de dados

3.3.1 Na nuvem (VM Oracle), uma única vez

1) Crie um subdomínio no DuckDNS apontando para o IP público da VM.
2) No console da Oracle, libere as portas TCP 80, 443 e 8883 (Security List). Não abra 5432 nem 1883.
3) Na VM, libere as mesmas portas no firewall do Ubuntu:

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 8883 -j ACCEPT
sudo netfilter-persistent save
```

4) Instale o Docker e baixe o código:

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2 git openssl
sudo usermod -aG docker $USER      # sair e entrar de novo no SSH
git clone https://github.com/oliveiravitor32/TCC2.git && cd TCC2/Codigo
```

5) Gere os segredos (senhas, chave da API, certificados e usuários do broker). O script cria o arquivo `.env` e mostra o que vai para o `segredos.h`:

```
./deploy/preparar.sh tcc-iluminacao-publica.duckdns.org
```

6) Suba os serviços e confira:

```
docker compose -f docker-compose.prod.yml --profile bairro up -d --build
docker compose -f docker-compose.prod.yml ps
curl https://tcc-iluminacao-publica.duckdns.org/pontos
```

Na primeira subida, o contêiner do PostgreSQL executa automaticamente o DDL `db/schema.sql`, as classes P3 a P6 de `db/seed.sql` e o cadastro de trechos e postes de `db/seed_cadastro.sql`. Num banco que já existe, as mudanças de estrutura são aplicadas pelas migrações numeradas de `db/migracoes/` (001 a 011), sempre com backup antes:

```
docker compose -f docker-compose.prod.yml exec -T db pg_dump -U iluminacao
iluminacao > ~/backup_$(date +%F).sql
docker compose -f docker-compose.prod.yml exec -T db psql -U iluminacao -d
iluminacao < db/migracoes/011_grupos_e_mapa.sql
```

A calibração da bancada é carregada uma vez com `deploy/calibracao_bancada.sql`. **Nunca use docker compose down -v**: o `-v` apaga o volume do banco.

3.3.2 Variáveis de ambiente

O arquivo `.env` é gerado pelo `preparar.sh` e fica só na VM. O quadro a seguir serve de `.env.example` (os valores reais nunca vão para o repositório).

Quadro 9 – Variáveis de ambiente (.env)

Variável	Para que serve	Exemplo
DOMINIO	Nome DuckDNS usado pelo Caddy e nos certificados	tcc-iluminacao-publica.duckdns.org
SENHA_BANCO	Senha do usuário `iluminacao` do PostgreSQL	(gerada, 32 caracteres)
CHAVE_API	Chave exigida no cabeçalho `X-Chave-Api` do `POST /eventos` e dos endpoints `/bancada`	(gerada)
CHAVE_DEMO	Chave pública do painel de demonstração (só eventos; revogar após a banca)	(gerada) ou vazia
SENHA_MQTT_SERVICO	Senha do usuário `servico` no broker (serviço de decisão)	(gerada)
SENHA_MQTT_NO	Senha do nó BANCADA-01 no broker (vai para o segredos.h)	(gerada)
SENHA_MQTT_FROTA	Senha da frota simulada (perfil bairro)	(gerada por deploy/novo_no.sh FROTA-SIM)
ORIGENS_PAINEL	Endereços autorizados a chamar a API pelo navegador (CORS)	https://seu-painel.vercel.app
MODO_TESTE	1 liga o "forçar evento imediato" do painel (só demonstração)	vazio ou 1
CICLO_NOITE_MIN	Duração, em minutos, da noite sintética do gerador de volumes	60

Fonte: Elaborado pelos autores (2026).

3.3.3 No computador (ambiente local de desenvolvimento)

Com o Docker Desktop aberto, no Git Bash, pasta `Codigo`:

```
./subir.sh            # banco, broker, API, decisão, nó simulado e painel
./subir.sh --sem-no   # idem, usando o ESP32 real
./parar.sh            # encerra tudo
```

A API fica em http://localhost:8000/docs (documentação interativa) e o painel em http://localhost:5173.

3.4 Instalação da aplicação cliente (painel)

1) No computador, pasta `Codigo/painel`: `npm install` e depois `npm run dev` (desenvolvimento) ou `npm run build` (gera a pasta `dist`).
2) Para apontar o painel para a VM, crie a variável `VITE_API_URL=https://tcc-iluminacao-publica.duckdns.org` antes do build.
3) Publicação: na Vercel, importe o repositório, escolha Root Directory = `Codigo/painel` (framework Vite) e cadastre `VITE_API_URL` em Environment Variables. Cada push na branch principal publica de novo.

4) Na VM, troque `ORIGENS_PAINEL` no `.env` pelo endereço gerado pela Vercel e rode `docker compose -f docker-compose.prod.yml up -d`.

O painel não exige instalação no computador do usuário: basta um navegador atualizado (Chrome, Edge ou Firefox). Ele só lê a API e se atualiza a cada 2 s.

4 SEÇÃO ESPECIAL – MÓDULO DE INTELIGÊNCIA ARTIFICIAL

4.1 Objetivo operacional da IA

O módulo de IA tem duas partes, ambas no serviço de decisão da nuvem:

- **Previsão (regressão):** um modelo XGBoost estima a **movimentação de pedestres da próxima hora** em cada trecho, como fração da referência do trecho (0 a 1), refeita a cada 15 min. Serve para não apagar cedo demais quando o movimento vai voltar e como segunda evidência contra falha da contagem das câmeras.
- **Decisão (inferência nebulosa Mamdani):** sete regras (base `v4-volume`) combinam volume observado, movimentação prevista, ocorrência, faixa horária e classe da via e escolhem um degrau de classe (P3, P4 ou P5).

Toda saída da IA passa depois pela **camada de verificação**, que não é IA: é código determinístico que impõe o piso e a taxa da NBR 5101. Assim, nenhum erro do modelo consegue deixar a via abaixo da norma. Há também o **diagnóstico** (detecção de anomalias por regra): compara a luz medida com a esperada e abre alertas de falha total ou perda de controle.

4.2 Dataset de treinamento e validação

4.2.1 Origem dos dados

São usados **datasets públicos** de contagem de pedestres, porque as câmeras de São Paulo não publicam esse dado hoje:

- **Modelo em operação:** contagens de pedestres e ciclistas a cada 15 min da cidade de Zurique (portal de dados abertos da Stadt Zürich, licença CC0; arquivo baixado em 30/09/2026, SHA-256 34a41dce...). Pedestres = soma das passagens nos dois sentidos.
- **Primeiro treino (histórico):** série horária de pedestres da cidade de Melbourne (City of Melbourne Open Data). Ficou como comparação.
- **Dados sintéticos:** a suíte de cenários (`Codigo/cenarios/`) e o gerador de volumes da demonstração de bairro produzem dados inventados apenas para testar o sistema; **eles nunca entram em treino nem em resultado do modelo**.

- **Sensores reais:** o sensor BH1750 do nó mede iluminância (lux) em tempo real; esse dado alimenta o diagnóstico, não o modelo de previsão.

4.2.2 Volume amostral e pré-processamento

Quadro 10 – Volume amostral e preparação da série de Zurique

Item	Valor
Janela	01/01/2016 a 31/12/2019, 14 pontos de contagem (12 com dados no ano de teste)
Registros lidos na janela	5.243.407 (2.804.555 nulos descartados; nenhum negativo)
Intervalos de 15 min na grade	1.642.752 (16.304 ausentes, não preenchidos)
Sensor travado	121 sequências (5.792 intervalos) de zero por 8 h ou mais em horário diurno, tratadas como ausentes
Horário de verão	Carimbos repetidos (114 linhas, 27/10/2019) combinados pela média
Linhas finais	1.168.268 de treino e 404.211 de teste

Fonte: Elaborado pelos autores (2026).

Normalização: a contagem de cada ponto é dividida pelo seu percentil 99, calculado **só no período de treino**, e limitada a 1. Assim, trechos movimentados e vazios ficam na mesma escala, e a referência de cada trecho no sistema segue a mesma lógica (p99 de 28 dias).

Engenharia de atributos (17): atrasos de 1, 2, 3, 4, 8, 96 (um dia) e 672 (uma semana) intervalos; médias móveis de 1 h, 3 h e 24 h; hora; quarto de hora; seno e cosseno da fração do dia; dia da semana; fim de semana; feriado do cantão de Zurique. O alvo é o nível médio dos próximos 60 min.

4.3 Arquitetura do modelo e treinamento

Algoritmo: reforço de gradiente com árvores de decisão (XGBoost, Chen e Guestrin, 2016), via `xgboost.XGBRegressor`. **Justificativa:** dados tabulares com atributos de calendário e atraso, em que modelos de árvores são a referência da literatura; treino rápido em CPU; previsão de milissegundos; e o modelo foi comparado, com regra de decisão fixada antes, contra regressão linear, floresta aleatória (Random Forest) e três referências sem treino (persistência, média sazonal e semana anterior). A inferência Mamdani foi escolhida por gerar regras legíveis, que o operador consegue auditar (RNF05).

Quadro 11 – Hiperparâmetros e divisão dos dados

Item	Valor
n_estimators	2000 (melhor iteração: 1999)
learning_rate	0,05
max_depth	8
subsample	0,8
colsample_bytree	0,8
min_child_weight	5
tree_method	hist
Parada antecipada	50 rodadas sem melhora no MAE de validação
Divisão treino/teste	Cronológica: treino de 2016 a 2018, teste = 2019 inteiro
Validação	Últimos 10 % do treino (hold-out cronológico) para a parada antecipada
Random Forest (comparação)	200 árvores, min_samples_leaf = 5, max_features = 0,5
Script e protocolo	`Codigo/modelo/treinar_zurique.py` e `PROTOCOLO_MODELO_ZURIQUE.md` (fixado antes do treino)

Fonte: Elaborado pelos autores (2026).

Por que não há validação cruzada aleatória (k-fold): em série temporal, embaralhar as linhas deixa o modelo ver o futuro no treino. Por isso usou-se partição cronológica com validação no fim do treino; a robustez foi conferida por ponto de contagem e por reamostragem de dias (bootstrap) no ano de teste.

4.4 Resultados e métricas empíricas

Como o modelo é de **regressão**, as métricas principais são o erro absoluto médio (MAE) e a raiz do erro quadrático médio (RMSE). Para ler a previsão como decisão, ela também foi transformada em classes (faixa baixa, média ou alta, pelos tercis 0,0693 e 0,2946), o que permite calcular acerto da faixa e F1.

Tabela 1 – Métricas no ano de teste (2019), série de Zurique em 15 min

Modelo	MAE	RMSE	MAE à noite	Acerto da faixa à noite	F1 "vai subir" à noite
XGBoost (em operação)	0,0416	0,0669	0,0228	91,4 %	0,638
Random Forest	0,0429	0,0694	0,0231	91,2 %	0,630
Regressão linear	0,0633	0,0972	0,0315	87,4 %	0,447
Semana anterior	0,0708	0,1277	0,0357	87,2 %	0,583
Média sazonal	0,0688	0,1140	0,0390	84,3 %	0,467
Persistência	0,0763	0,1220	0,0469	82,8 %	0,269

Fonte: Elaborado pelos autores (2026), a partir de Codigo/modelo/resultados/metricas_zurique.json.

O XGBoost teve o menor erro em 12 dos 12 pontos contra a regressão linear e as referências simples, e em 11 dos 12 contra a Random Forest. Pelo critério fixado antes (superar a melhor referência simples em ao menos 2 pontos percentuais no acerto da faixa e 0,02 no F1), ele **não é substituível** por uma regra simples na resolução de 15 min. Com a mesma série somada por hora, a vantagem desaparece; isso é declarado no TCC (Tabela 4 do texto).

Para atender ao pedido de matriz de confusão, a previsão contínua foi convertida nas três faixas, no período noturno (18h às 6h) do ano de teste, com o modelo salvo. A soma da diagonal dividida pelo total reproduz o acerto da faixa da Tabela 1 (91,4 %).

Tabela 2 – Matriz de confusão das faixas previstas (XGBoost, período noturno de 2019, 201.360 casos)

Real Previsto	Baixa	Média	Alta
Baixa	123.389	6.519	33
Média	4.430	42.319	3.379
Alta	17	2.900	18.374

Fonte: Elaborado pelos autores (2026), com o modelo modelo_previsao_zurique.json.

Tabela 3 – Precisão, revocação e F1 por faixa (período noturno de 2019)

Faixa	Precisão	Revocação (recall)	F1
Baixa	0,965	0,950	0,957
Média	0,818	0,844	0,831
Alta	0,843	0,863	0,853

Fonte: Elaborado pelos autores (2026).

Os erros ficam quase todos entre faixas vizinhas: só 50 casos (0,02 %) confundem baixa com alta. A faixa média é a mais difícil, o que é esperado, porque fica entre as duas fronteiras.

Resultado no sistema inteiro (TCC, Capítulo 4): em 667.200 passos verificados não houve violação do piso nem da taxa da norma; a latência entre o evento e a luz medida no nó foi de 1,03 s a 1,47 s; o regime adaptativo consumiu 24,3 % menos energia que o fluxo pleno na suíte de cenários e 37,6 % menos em noites reais. A hipótese de que a previsão antecipa a atenuação (H3) foi rejeitada: com previsão, 27,9 % da janela de baixa demanda ficou em transição, contra 23,3 % sem ela.

4.5 Declaração formal de uso de IA assistiva

Os autores declaram que empregaram modelos de linguagem generativa (**Claude Opus 5 e Claude Opus 5.5, da Anthropic, pela ferramenta Claude Code**) como apoio na codificação (firmware, serviços Python, painel e scripts), na diagramação (figuras geradas por script, incluindo as Figuras 1 e 3 deste guia) e na redação e revisão da documentação, inclusive deste guia. O uso de IA no TCC está detalhado, seção por seção, no Apêndice A do texto (Quadro 19). Todo artefato foi revisado, testado e conferido pelos autores, que **assumem inteira responsabilidade técnica** pelo hardware, pelo software, pelos dados e pelos resultados aqui apresentados.

5 MANUAL DE OPERAÇÃO E DIAGNÓSTICO DO CONJUNTO PROTOTIPADO

5.1 Procedimento de partida e calibração

1) Confira que os serviços da nuvem estão no ar: abra https://tcc-iluminacao-publica.duckdns.org/pontos no navegador (deve voltar um JSON) ou rode `docker compose -f docker-compose.prod.yml ps` na VM (db, broker, api, decisao e caddy como "Up").
2) Ligue a fonte de 12 V na tomada (a fita ainda fica apagada, porque o ESP32 não está gerando PWM).
3) Ligue o ESP32 pela USB. Ao energizar, o nó vai para o **nível pleno** (nunca parte apagado).
4) Verifique o handshake no Monitor Serial (115200 baud): conexão Wi-Fi, conexão TLS ao broker e publicação de "online" no tópico de estado. No painel, o ponto BANCADA-01 passa a aparecer como comunicando (verde no mapa).
5) Feche a tampa da caixa e rode a conferência de calibração (cerca de 1 min), no notebook, pasta `Codigo`:

```
.venv\Scripts\python -m bancada.conferir --vm
```

A conferência coloca o nó em modo manutenção pela serial (o painel mostra o ponto em cinza), mede a **leitura de repouso** com a fita apagada (residual; a sessão é descartada se passar de 5 % do valor da classe, 0,5 lux), mede os níveis pleno (10 lux), intermediário e piso (5 lux) e compara com a tolerância do diagnóstico. Se aprovar, abre a sessão de medição no banco da VM; se reprovar, recalibra e grava a nova tabela duty × lux na memória do ESP32, sem recompilar. Ao terminar, o nó sai da manutenção e volta a aceitar comandos.

6) Opcional: para ver a frota do bairro em movimento, ligue o gerador de volumes na VM (`docker compose -f docker-compose.prod.yml --profile bairro up -d gerador`).

5.2 Instruções de operação diária

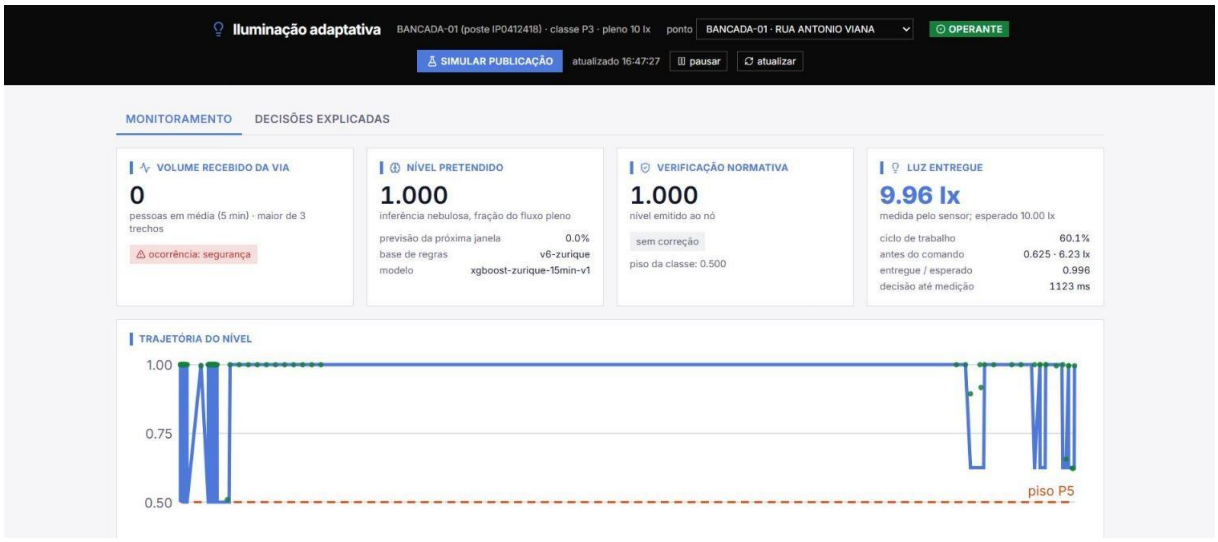
O operador interage com o sistema **pelo painel web**. Ele não comanda as luzes à mão: a decisão é automática, e o painel serve para acompanhar, auditar e receber alertas. As abas são:

Quadro 12 – Abas do painel e como ler cada uma

Aba	O que mostra	Como interpretar
Monitoramento	Por ponto: nível pretendido pela IA, nível emitido após a verificação, duty aplicado e lux medido, em gráfico ao longo do tempo	As linhas de nível e de lux devem andar juntas. A descida é em rampa (no máximo 0,1 do pleno por minuto); a subida é imediata. O nível nunca fica abaixo de 0,5 (piso P5)
Mapa do bairro	Os 35 postes do Jardim Helena sobre o mapa, ligados aos 16 trechos de via	O preenchimento mostra a saúde (verde = ok, amarelo = alerta, vermelho = sem comunicação, cinza = manutenção); a borda mostra o nível de luz
Alertas	Um cartão por situação atual: falha total, perda de controle, sem comunicação, ponto em falha, manutenção; contador vermelho na aba	Cada cartão traz o ponto, o grupo, desde quando, a razão medida/esperada e o botão "ver no monitoramento"
Decisões explicadas	Para cada decisão: eventos usados, previsão, regras ativadas, nível pretendido, restrição aplicada e nível emitido	Responde "por que a luz está assim". Ex.: R5 (observada baixa e prevista baixa) → classe mínima, limitada pela taxa
Consumo	Energia estimada do regime adaptativo contra o fluxo pleno e o horário fixo	Estimativa do protótipo (luminária de referência de 40 W); não é economia anual nem em reais

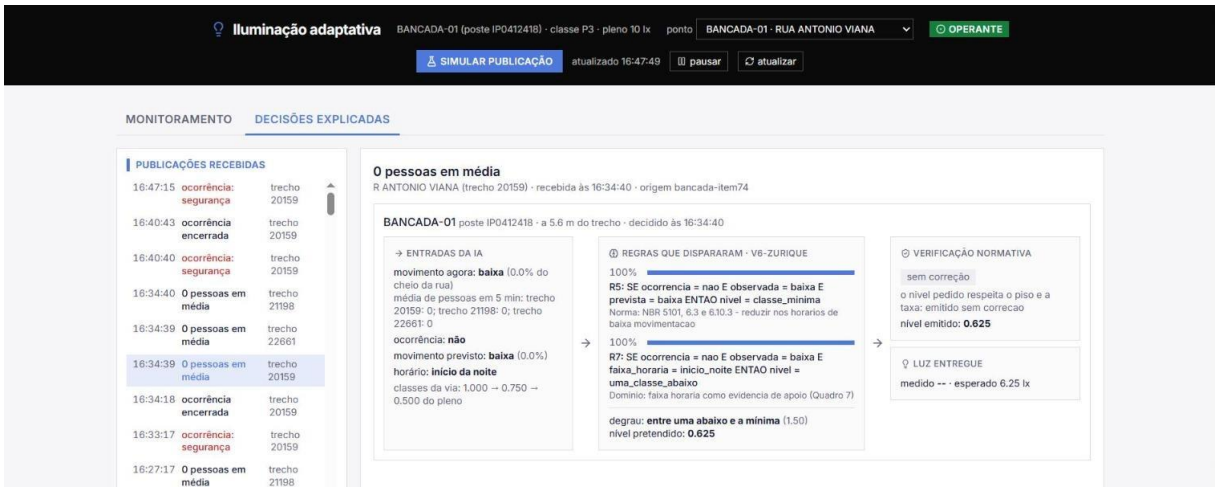
Fonte: Elaborado pelos autores (2026).

Figura 4 – Aba de monitoramento do painel



Fonte: Captura do painel do protótipo (2026).

Figura 5 – Decisão explicada pelo painel



Fonte: Captura do painel do protótipo (2026).

Para demonstrar a reação do sistema, use o botão "simular leitura" do painel ou, no Git Bash, publique um volume baixo: `./evento.sh volume 0.05`. A luz deve descer em rampa até o piso. Uma ocorrência (`./evento.sh ocorrencia seguranca`) faz a luz voltar ao pleno na hora.

5.3 Guia de resolução de problemas

Quadro 13 – Sintoma, causa provável e procedimento corretivo

Categoria	Sintoma	Causa provável	Procedimento corretivo
Rede	Serial mostra tentativas de Wi-Fi sem conectar	Rede de 5 GHz, senha errada ou sinal fraco	Usar rede de 2,4 GHz; conferir WIFI_SSID e WIFI_SENHA no segredos.h e regravar
Rede	Ponto vermelho (sem comunicação) no painel	Nó desligado, Wi-Fi caiu ou broker fora; o broker publicou o testamento "offline"	Conferir a USB e o Wi-Fi; na VM, `docker compose ... ps` e `logs broker`. Sem comando, o nó volta sozinho ao pleno (failsafe)
MQTT	Wi-Fi conecta, mas o broker recusa (erro de TLS ou de autenticação)	Certificado da CA errado, senha do nó errada ou porta 8883 fechada	Copiar de novo o certificado e a senha mostrados pelo preparar.sh; conferir a regra de ingresso 8883 na Oracle e o iptables
MQTT	Comandos chegam atrasados ou se perdem	Rede instável; QoS 1 reenvia, mas o comando pode vencer	Aproximar o roteador; o nó aplica o próximo comando válido e, vencida a validade (até 15 min), vai ao pleno. Ver latência na aba de monitoramento
Sensor	Lux = 0 ou leitura fixa; serial acusa erro do BH1750	Mau contato em SDA/SCL ou VCC (já ocorreu por solda fria)	Desligar, refazer as ligações GPIO21/22 e 3V3; ressoldar o conector do GY-302
Sensor	Leitura fora da faixa (lux alto com fita apagada)	Tampa aberta ou luz entrando pelos furos (residual > 5 %)	Fechar e vedar a caixa; rodar `bancada.conferir` de novo

Categoria	Sintoma	Causa provável	Procedimento corretivo
Atuador	Fita apagada com comando > 0; alerta "falha total"	Fonte de 12 V desligada, GND não comum ou fio solto na saída do D4184	Conferir a fonte, o GND comum e os bornes OUT+/OUT−. O alerta fecha sozinho quando a leitura volta ao normal
Atuador	Alerta "perda de controle" (luz diferente da esperada por vários ciclos)	Caixa mexida, difusor deslocado ou calibração vencida	Recolocar o difusor e rodar `bancada.conferir`, que recalibra se reprovar
Servidor	API responde 401	Falta ou erro no cabeçalho `X-Chave-Api`	Usar a CHAVE_API do .env (ou a CHAVE_DEMO no painel)
Servidor	API responde 422	Campo fora do vocabulário (tipo, trecho inexistente, pessoas_media < 0)	Corrigir o JSON conforme `GET /docs`; num pacote, só o item errado é recusado
Servidor	Painel não carrega dados (erro de CORS)	ORIGENS_PAINEL não inclui o endereço do painel	Ajustar o .env e subir de novo a API
Servidor	Disco da VM cheio ou serviço reiniciando	Banco cresceu ou falta de memória	`docker system df`; os logs já são limitados a 5 × 10 MB por serviço; ver `docker compose ... logs decisao`

Fonte: Elaborado pelos autores (2026).

5.4 Desligamento seguro e manutenção preventiva

5.4.1 Desligamento

1) Desligue primeiro a fonte de 12 V (a fita apaga) e depois retire a USB do ESP32. O broker publica "offline" e o painel mostra o ponto em vermelho, o que é o esperado.
2) Os serviços da nuvem podem ficar ligados. Para pará-los: `docker compose -f docker-compose.prod.yml down` (mantém o banco). **Nunca usar down -v.**
3) No ambiente local: `./parar.sh`.

5.4.2 Manutenção preventiva

Quadro 14 – Rotinas de manutenção preventiva

Rotina	Frequência	Procedimento
Limpeza do sensor e do difusor	Antes de cada sessão	Pano seco e macio na janela do BH1750; conferir que o difusor está no lugar e sem poeira
Conferência da calibração	Antes de cada sessão ou demonstração	`python -m bancada.conferir --vm`
Inspeção das ligações	Mensal ou após transporte	Conferir jumpers na protoboard, bornes do D4184 e soldas do GY-302

Rotina	Frequência	Procedimento
Atualização do firmware	Quando o código do nó mudar	Regravar pela Arduino IDE (seção 3.2); o segredos.h e a tabela de calibração na memória do ESP32 são mantidos
Atualização dos serviços	A cada versão	Na VM: `git pull` e `docker compose -f docker-compose.prod.yml --profile bairro up -d --build`
Backup do banco	Antes de qualquer migração e periodicamente	`pg_dump` (seção 3.3.1); guardar fora da VM junto com o .env e os certificados do broker
Segurança	Após a banca	Desligar MODO_TESTE e revogar a CHAVE_DEMO no .env

Fonte: Elaborado pelos autores (2026).

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR 5101**: iluminação pública: procedimento. Rio de Janeiro: ABNT, 2018.

CHEN, T.; GUESTRIN, C. XGBoost: a scalable tree boosting system. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 22., 2016, San Francisco. **Proceedings** [...]. New York: ACM, 2016. p. 785-794.

CITY OF MELBOURNE. **Pedestrian counting system**. Melbourne: City of Melbourne Open Data, 2026. Disponível em: https://data.melbourne.vic.gov.au. Acesso em: 2026.

DACAL, B. B.; VERGARA, L. A.; SANTOS, V. O. **Controle inteligente da iluminação pública a partir de dados urbanos já existentes**: proposta de integração entre plataforma de monitoramento e telegestão no município de São Paulo. 2026. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) – Universidade Paulista, São Paulo, 2026.

STADT ZÜRICH. **Daten der automatischen Fussgänger- und Velozählung**. Zürich: Open Data Zürich, 2026. Disponível em: https://data.stadt-zuerich.ch. Acesso em: 30 set. 2026.